

Manual Técnico — Herramienta de Transparencia Algorítmica

- Manual Técnico
 - Herramienta de Transparencia Algorítmica — GobLab UAI
 - Índice
 - 1. Arquitectura general
 - 2. Estructura del repositorio
 - 3. El pipeline de contenido: del Excel a `sections.ts`
 - 4. Modelo de datos del cuestionario
 - 5. El motor de visibilidad
 - 6. Contexto normativo (Chile / Internacional)
 - 7. Persistencia de respuestas
 - 8. Generación del documento y del PDF
 - 9. Sistema de diseño «Civic Rose»
 - 10. Supabase: feedback, encuesta y registro
 - 11. Analítica (GA4)
 - 12. Variables de entorno
 - 13. Ejecución local
 - 14. Verificación y despliegue
 - 15. Decisiones de diseño
 - 16. Cómo extender la herramienta

Manual Técnico

Herramienta de Transparencia Algorítmica — GobLab UAI

Documentación para desarrolladores y mantenedores: arquitectura, el pipeline que genera el cuestionario desde el Excel del Drive, el motor de condicionales, la

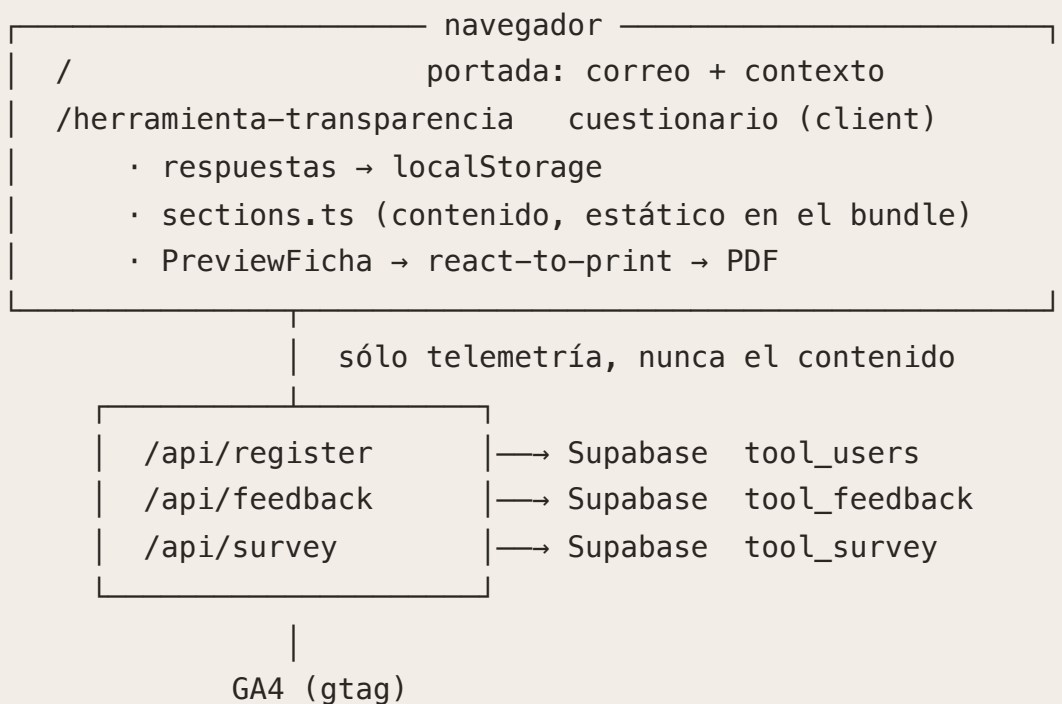
generación del PDF, persistencia, ejecución local y despliegue. El manual de uso funcional está en [MANUAL_USUARIO.md](#).

Índice

1. [Arquitectura general](#)
 2. [Estructura del repositorio](#)
 3. [El pipeline de contenido: del Excel a `sections.ts`](#)
 4. [Modelo de datos del cuestionario](#)
 5. [El motor de visibilidad](#)
 6. [Contexto normativo \(Chile / Internacional\)](#)
 7. [Persistencia de respuestas](#)
 8. [Generación del documento y del PDF](#)
 9. [Sistema de diseño «Civic Rose»](#)
 10. [Supabase: feedback, encuesta y registro](#)
 11. [Análítica \(GA4\)](#)
 12. [Variables de entorno](#)
 13. [Ejecución local](#)
 14. [Verificación y despliegue](#)
 15. [Decisiones de diseño](#)
 16. [Cómo extender la herramienta](#)
-

1. Arquitectura general

Una sola aplicación **Next.js 15 (App Router) + TypeScript + Tailwind**, desplegada en Vercel. **No hay backend propio ni base de datos de la aplicación:** el contenido de las fichas nunca sale del navegador.



Las tres rutas de API son delgadas a propósito: reciben un JSON, lo insertan en Supabase vía PostgreSQL con la anon key y responden. No hay ORM ni cliente de Supabase — sólo `fetch`. La anon key sólo tiene permiso de INSERT (ver [sección 10](#)).

El cuestionario es 100% cliente. `sections.ts` se compila dentro del bundle, así que no hay llamada de red para obtener las preguntas.

2. Estructura del repositorio

src/	
app/	
layout.tsx	fuentes (Inter/Fraunces/JetBrains Mono)
page.tsx	portada: hero, grafo de dimensiones,
herramienta-transparencia/	
page.tsx	guard: sin userEmail → redirige a /
api/	
register/route.ts	→ tool_users
feedback/route.ts	→ tool_feedback (con columnas de con
survey/route.ts	→ tool_survey (con respaldo a tool_f
components/	
transparency-tool.tsx	el cuestionario completo (712 líneas)
preview-ficha.tsx	el documento imprimible (280 líneas)
PdfExportButton.tsx	react-to-print + hoja @page
FeedbackPill.tsx	pill flotante de feedback
QuestionFeedback.tsx	👍/👎 + motivos, por pregunta
SatisfactionSurvey.tsx	encuesta de satisfacción
civic-icons.tsx	iconos de trazo del sistema de diseñ
ui/	primitivas shadcn/radix (toast, sele
data/	
sections.ts	GENERADO · 12 dimensiones, 112 pregu
question-types.ts	tipos + motor de visibilidad (a mano
survey.ts	enunciados de la encuesta
lib/	
civic.ts	tokens del sistema de diseño
contexto.ts	contexto normativo
analytics.ts	eventos GA4 + registro de usuario
feedback.ts	cliente de /api/feedback y /api/surv
styles/print-fixes.css	columnas y cortes del documento
docs/	
fta-gen/	pipeline de contenido (ver sección 3
manual/	este manual y el de usuario
supabase/migrations/	SQL idempotente, se corre a mano

Regla importante: `src/data/sections.ts` es generado y no se edita a mano. Todo lo demás sí.

3. El pipeline de contenido: del Excel a `sections.ts`

El contenido del cuestionario lo mantiene el equipo de contenido en un Excel en Drive (*FTA Gen (Hito1) Contenido/Desarrollo*). El pipeline lo convierte en código:

```
Excel (Drive)
  | docs/fta-gen/extraer-excel.py      (Python + openpyxl)
  ▼
docs/fta-gen/contenido-hito1.json      transcripción fiel, sin c
  | docs/fta-gen/generar-sections.mjs (Node)
  ▼
src/data/sections.ts                  tipado, con ids estables
```

`extraer-excel.py` — transcripción

Sólo transcribe. Resuelve las rarezas del Excel y nada más:

- **Las columnas cambian de hoja en hoja** → se mapean por encabezado, no por posición.
- **Excel convirtió los IDs en fechas** (1.4 → 2026-01-04) → se reconstruyen como `mes.día`.
- **- y Se mantiene son marcadores, no contenido** → se normalizan a vacío.
- La columna *Contenido nuevo* se usa de dos formas en el documento (reemplazo real y comentario suelto), así que **manda *Contenido actual* cuando trae algo**. Verificado sobre las 12 hojas.

`generar-sections.mjs` — todo el criterio

Acá vive cada decisión, explícita y revisable en un solo lugar en vez de repartida por 112 objetos:

Bloque	Qué decide
DIMS	Slug y título definitivo de cada dimensión
TIPO	Excepciones de tipo de campo (por defecto es <code>textarea</code>)
MULTISELECT	Qué listas de alternativas admiten varias marcas
CONDICIONES	El grafo completo de condicionales
SI_NO_POR_CONDICION	Preguntas sí/no que el Excel no marcó como tales, inferidas de las condiciones que las evalúan

Numeración. Cada dimensión se renumera **por posición** → **D.N** (4.1 ... 4.35). El Excel mezcla dos criterios de numeración y renumerar por posición es lo único que hace consistentes todas las referencias cruzadas.

Identificadores. `d04_q12`: dimensión y posición, con ceros a la izquierda. Son estables e independientes del número visible, que cambia cada vez que se agrega o quita una pregunta. El número visible va aparte, en `numero`, y es el que viaja en el feedback.

Variantes internacionales. El Excel las agrega al final de cada hoja con la columna `Versión = Internacional`. El generador las aparta antes de numerar (si entraran al recorrido, correrían la numeración de todas las demás) y las empareja por ID del Excel para emitir las como `overrides.internacional`.

Validaciones del generador

Al regenerar, avisa de:

1. **`dependsOn` hacia un id inexistente** — la pregunta nunca se mostraría.
2. **Ciclos** en el grafo de dependencias.
3. **Dependencias que crucen de dimensión** — romperían el índice de visibilidad, que es por dimensión (ver [sección 5](#)).
4. **`PREGUNTAS_CLAVE` apuntando a ids que ya no existen** — el sidebar y el encabezado del PDF los referencian directamente, y una referencia obsoleta no falla: deja el campo vacío. Ya pasó una vez.
5. Informativos: preguntas sin tooltip, alternativas repetidas, conteo de cadenas profundas.

```
node docs/fta-gen/generar-sections.mjs
```

Es **determinista**: correrlo sin cambios de contenido deja `sections.ts` byte-idéntico. Si el diff no está vacío tras una corrida limpia, algo cambió en el JSON o en el criterio.

4. Modelo de datos del cuestionario

`src/data/question-types.ts` define el modelo y **no se genera**.

```
interface Question {
  id: string           // 'd04_q12' - estable
  numero: string       // '4.12'    - visible, cambia con el
                        contenido
  text: string
  type: 'text' | 'textarea' | 'radio' | 'select' | 'multiselect' |
        'slider' | 'date'
  options?: string[]
  isRequired: boolean
  tooltip: string
  placeholder?: string
  bloque?: string      // subtítulo dentro de la dimensión
                        (lo usa Legal)
  dependsOn?: Condition[] // se muestran si se cumplen TODAS
  iaGen?: boolean       // subconjunto de IA generativa
  soloContexto?: Contexto // exclusiva de un contexto normativo
  overrides?: Partial<Record<Contexto, { text?, tooltip?,
                                options?, placeholder? }>>
}

interface Condition {
  questionId: string
  equals?: string | number // radio, select
  includes?: string        // multiselect
  atLeast?: number         // slider
}
```

`dependsOn` es un **array con semántica AND**. Un OR requeriría extender el tipo; hoy no hace falta.

Cifras actuales: 12 dimensiones, 112 preguntas, 92 obligatorias, 53 con condicional, profundidad máxima de cadena 3.

5. El motor de visibilidad

Es la parte más delicada del código. Vive en `question-types.ts` y **la comparten el cuestionario y el PDF a propósito**: si cada uno decidiera por su cuenta qué pregunta aplica, el documento mostraría respuestas de preguntas ocultas.

`shouldShow` es transitivo

No basta con que el padre tenga el valor esperado: **el padre además tiene que estar visible**.

```
shouldShow(question, answers, contexto, index?, seen?)
```

La razón es que **las respuestas nunca se purgan** — decisión deliberada, para que revertir un cambio no borre lo escrito. Sin evaluación transitiva, una respuesta huérfana mantiene viva toda su rama:

```
5.1 = Sí    → abre 5.2
5.2 = Sí    → abre 19 obligatorias de Ciberseguridad
5.1 → No    → 5.2 se oculta, pero su 'Sí' sigue en formData
               → sin transitividad, las 19 siguen bloqueando la desca
               y entran al PDF preguntas que nadie vio
```

De las 112 preguntas, **26 están en cadenas de profundidad ≥ 2 y las 26 son obligatorias** (19 cuelgan de `d05_q02`).

El índice es por dimensión

`shouldShow` necesita resolver el padre, así que recibe un `Map<id, Question>`. El índice se construye **por sección**, memoizado en un `WeakMap`:

```
const indicePorSeccion = new WeakMap<Section, Map<string,
  Question>>()
```

Por qué por sección y no global: todas las dependencias del cuestionario son intra-dimensión. Un índice global obligaría a importar `sections.ts` desde `question-types.ts`, y `sections.ts` ya hace `export * from './question-types'` — sería un ciclo. El generador avisa si alguien escribe una dependencia cruzada, que es lo que rompería este supuesto.

Corte de ciclos

`seen: Set<string>` corta la recursión. Ante un ciclo **devuelve true**: un error de contenido no debe ocultar preguntas en silencio; mejor mostrar de más.

API pública

Función	Qué hace
<code>shouldShow(q, answers, contexto, index?, seen?)</code>	¿Aplica esta pregunta?
<code>forContexto(q, contexto)</code>	Aplica los <code>overrides</code> del contexto
<code>visibleQuestions(section, answers, contexto)</code>	Filtra + resuelve contexto. Única puerta de entrada
<code>isAnswered(q, answers)</code>	Trata correctamente el multiselect vacío
<code>sectionProgress(section, answers, contexto)</code>	% 0–100
<code>isSectionComplete(section, answers, contexto)</code>	Sin obligatorias pendientes y con al menos una respuesta

contexto es obligatorio a propósito. Podría tener valor por defecto, pero entonces un componente que olvidara pasarlo mostraría el contenido chileno sin fallar. Así el compilador señala cada llamada.

Consumidores

Los cuatro usan `visibleQuestions`, nunca `section.questions` directo:

Dónde	Para qué
<code>transparency-tool.tsx · pendientes</code>	Obligatorias visibles sin responder
<code>transparency-tool.tsx · progress</code>	Avance global
<code>transparency-tool.tsx · visibleInSection</code>	Render del formulario
<code>preview-ficha.tsx · dimensiones</code>	Contenido del documento

6. Contexto normativo (Chile / Internacional)

`src/lib/contexto.ts`. Mismo modelo que la Evaluación de Impacto Algorítmico, para que ambas herramientas se comporten igual.

```
type Contexto = 'chile' | 'internacional'
const CONTEXTO_DEFAULT: Contexto = 'chile'
```

Propagación: se elige en la portada → se guarda en `localStorage` (`contextoFicha`) y se pasa por la URL (`?contexto=...`). El cuestionario da prioridad a la URL —permite compartir un enlace— y cae a lo guardado. `normalizarContexto()` convierte cualquier valor ausente o inválido al default, así que los enlaces antiguos siguen funcionando.

Dónde se ve: píldora en el encabezado del cuestionario, y **declarado en el encabezado y el pie de la ficha** — quien lea el documento debe saber contra qué marco normativo se respondió.

Contenido diferenciado: hoy 4 preguntas tienen `overrides.internacional` (4.9, 5.1, 5.2, 5.17), todas reescrituras que quitan la referencia a normativa chilena. Ninguna pregunta usa `soloContexto` todavía.

7. Persistencia de respuestas

Todo en `localStorage`, nada en servidor.

Clave	Contenido
<code>userEmail</code>	Correo con el que se inició
<code>answers_\${email}</code>	Objeto { <code>[questionId]: respuesta</code> }
<code>contextoFicha</code>	<code>'chile' 'internacional'</code>
<code>surveySent_\${email}</code>	<code>'1'</code> si ya respondió la encuesta

Se escribe en cada `handleInputChange`. No hay debounce: el volumen es pequeño y perder una respuesta por un cierre inesperado es peor que escribir de más.

Limitación conocida: las respuestas viven en ese navegador y ese equipo. No hay forma de consolidar el trabajo de varias personas en distintas máquinas. Está documentado en el manual de usuario como advertencia.

Las respuestas huérfanas se conservan. Al cerrarse una rama, sus respuestas quedan en `localStorage` pero ninguna función las lee. Revertir el cambio las recupera intactas.

8. Generación del documento y del PDF

El documento se recorre desde los datos

`preview-ficha.tsx` **no mapea campos a mano**: recorre `sections` y arma el documento con las preguntas visibles y respondidas.

```
const dimensiones = sections
  .map((section, i) => ({
    n: String(i + 1).padStart(2, '0'),
    title: section.title,
    respondidas: visibleQuestions(section, formData, contexto)
      .filter(q => isAnswered(q, formData)),
  }))
  .filter(d => d.respondidas.length > 0) // sin secciones vacías
```

Por qué importa. La versión anterior mapeaba campo por campo y perdía respuestas en silencio cuando un identificador no coincidía: la dimensión de Categorización salía casi vacía aunque la persona la hubiera respondido. Con 112 preguntas el mapeo manual es inmantenible.

El PDF sale por impresión, no por librería

`PdfExportButton` usa **react-to-print**: clona `#ficha-preview` en un `iframe`, le inyecta una hoja `pageStyle` y llama a `window.print()`. La persona elige *Guardar como PDF*.

No hay `html2pdf/js/pdf`. Se eliminaron porque no se usaban y arrastraban vulnerabilidades.

Consecuencias de diseño:

- **Los estilos van inline** en `preview-ficha.tsx`, con los tokens Civic Rose. El diálogo de impresión conserva los estilos calculados del nodo; las clases de Tailwind del

documento padre no llegan al iframe.

- **print-color-adjust: exact** es imprescindible. Sin él Chrome descarta los fondos y la ficha sale en blanco y negro.
- **@page** define A4 con márgenes de 18 mm laterales y el pie de tres líneas.
- **Las secciones fluyen entre columnas.** Con **break-inside: avoid-column**, Legal y Ciberseguridad —que no caben en una columna— empujaban páginas enteras en blanco. Sólo el `<h3>` evita quedar huérfano.
- **La vista previa usa las mismas dos columnas que el PDF**, para que lo que se aprueba sea lo que se descarga.

Verificar el PDF de verdad

La vista previa no prueba la paginación. Para comprobar cortes de página y columnas hay que **imprimir**:

```
# 1. Ruta temporal que monte PreviewFicha con un fixture y la hoja
    pageStyle
#   (sacando #ficha-preview del overlay fijo, como hace react-to-
    print)
# 2. Imprimir con Chrome headless
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
  --headless --disable-gpu --no-pdf-header-footer --virtual-time-
    budget=15000 \
  --print-to-pdf=/tmp/ficha.pdf "http://localhost:3000/<ruta-
    temporal>"
pdftinfo /tmp/ficha.pdf | grep Pages
pdftoppm -png -r 90 /tmp/ficha.pdf /tmp/pag # inspección visual
```

Con el contenido completo, la ficha son **3 páginas A4**.

9. Sistema de diseño «Civic Rose»

`src/lib/civic.ts` — tokens compartidos con la EIA.

T.ink / ink80 / ink60 / ink40 / ink20	escala de texto
T.paper / paperDeep / line	fondos y bordes
T.rose / roseLight / roseTint	rosa
T.burgundy / rosePaper	burdeos y su fondo
T.success / warn	
SERIF = Fraunces	titulares
MONO = JetBrains Mono	rótulos y cifras
inputBase	estilo base de todos los campos

Las fuentes se cargan en `layout.tsx` con `next/font/google` y se exponen como variables CSS. `civic-icons.tsx` trae los iconos de trazo, que heredan color vía `currentColor`.

Los componentes de la herramienta usan estilos inline con estos tokens, no clases de Tailwind. Tailwind queda para las primitivas de `ui/`. Es la convención heredada de la EIA y lo que hace que el documento imprima bien.

10. Supabase: feedback, encuesta y registro

Proyecto compartido con las demás herramientas del GobLab. La columna `tool` separa las filas de cada una; el valor viene de `SUPABASE_TOOL_NAME`.

Tabla	Qué guarda	Ruta
<code>tool_users</code>	{ email, tool_name }	<code>/api/register</code>
<code>tool_feedback</code>	Comentarios, con contexto de dónde se enviaron	<code>/api/feedback</code>
<code>tool_survey</code>	Encuesta de satisfacción, una columna por pregunta	<code>/api/survey</code>

RLS

La anon key **sólo puede insertar**. No hay política de `SELECT` porque `tool_survey` guarda datos de contacto. Consecuencia práctica: un `SELECT` con anon devuelve vacío aunque haya filas, y un `DELETE` responde 204 sin borrar nada. Para leer o limpiar hay que usar el panel de Supabase o una service role key.

Degradación

Las rutas están escritas para **no perder un envío por una columna que falte**:

- `/api/feedback` intenta insertar con las columnas de contexto (`pantalla`, `seccion`, `pregunta`, `question_id`, `progreso`) y **reintenta sin ellas** si la migración no se corrió.
- `/api/survey` intenta con `p1_institucion` y **reintenta sin ese campo**; si la tabla entera no existiera, cae a `tool_feedback` guardando la encuesta como texto.

Migraciones

`supabase/migrations/`, SQL idempotente que **se corre a mano** en el SQL Editor. No hay herramienta de migraciones.

Archivo	Qué hace
<code>001_tool_survey.sql</code>	Crea <code>tool_survey</code> y agrega las columnas de contexto a <code>tool_feedback</code>
<code>002_survey_institucion.sql</code>	Agrega <code>p1_institucion</code> a <code>tool_survey</code>

11. Analítica (GA4)

`src/lib/analytics.ts`. El script se carga en `layout.tsx` sólo si `NEXT_PUBLIC_GA_MEASUREMENT_ID` está definido.

Evento	Cuándo	Parámetros
tool_start	Al iniciar la ficha, desde la portada	tool_name
section_complete	Al avanzar de dimensión	section_name, section_index, progress_pct
tool_complete	Al abrir la vista previa	tool_name
tool_export	Al abrir la vista previa	format
question_feedback	👍 / 👎 en una pregunta	question_id, helpful
feedback_submit	Al enviar feedback o la encuesta	feedback_category, screen

tool_start se dispara **en la portada**, no al montar el cuestionario. En el montaje se emitía dos veces al volver con respuestas guardadas.

Los 👍 se registran sólo en GA4 y **no crean fila en Supabase**: son muchos eventos sin texto accionable, y en GA4 sirven igual para medir la proporción de preguntas poco claras.

12. Variables de entorno

Variable	Dónde	Para qué
SUPABASE_URL	Servidor	Endpoint de PostgREST
SUPABASE_ANON_KEY	Servidor	Autenticación de los inserts
SUPABASE_TOOL_NAME	Servidor	Valor de la columna tool. Default: herramienta de transparencia
NEXT_PUBLIC_GA_MEASUREMENT_ID	Cliente	ID de GA4. Sin él, no se carga gtag
NEXT_PUBLIC_VERSION	Cliente	Versión mostrada en portada y pie del PDF

NEXT_PUBLIC_* se incrusta en el build. Cambiarlas en Vercel **exige redespregar**; no se leen en tiempo de ejecución. Es la causa clásica de que la portada muestre una versión vieja con contenido nuevo.

13. Ejecución local

```
pnpm install
pnpm dev           # http://localhost:3000
```

Ojo con el gestor de paquetes. Localmente se usa **pnpm**, pero el repo commitea **package-lock.json** porque **Vercel compila con npm**. **pnpm-lock.yaml** está en **.gitignore** a propósito: si se subiera, Vercel cambiaría a pnpm e ignoraría el lock de npm.

Al agregar o subir una dependencia hay que regenerar **ambos**:

```
npm install --package-lock-only    # actualiza package-lock.json
                                   (sin tocar node_modules)
pnpm install                       # actualiza el node_modules
  local
```

El entorno usa **fnm**:

```
eval "$(fnm env)"; fnm use default
export PNPM_HOME=$HOME/Library/pnpm
```

next build rompe el dev server. Comparten **.next**, así que tras un build el servidor de desarrollo devuelve *Internal Server Error* con **ENOENT** de manifests. Se arregla reiniciándolo.

14. Verificación y despliegue

No hay suite de tests. La verificación es manual y por capas:

```
npx tsc --noEmit           # tipos
node docs/fta-gen/generar-secciones.mjs # contenido: avisos + diff
                                vacío
npx next build              # compila
```

Además, para cambios en el motor de visibilidad conviene reproducir a mano el escenario de Ciberseguridad: responder 5.1 = Sí y 5.2 = Sí, dejar alguna de las 19 hijas sin responder, y luego cambiar 5.1 = No. Deben desaparecer las 19, subir el avance, destrabarse la descarga y no aparecer en el PDF — y al volver a 5.1 = Sí, reaparecer con lo ya escrito.

Despliegue

Vercel, conectado al repositorio. **main va a producción**; cualquier otra rama genera un *preview* protegido por la autenticación de Vercel (para compartirlo con alguien sin cuenta hay que usar el botón *Share* del deployment).

```
gh api repos/johanpina/herramienta_transparencia_modelo/deployments
\
--jq '[0] | {environment, ref: .ref[0:8]}'
```

Vercel bloquea builds con vulnerabilidades altas en Next.js. Ya ocurrió una vez. Si un deploy se rechaza sin error de compilación, revisar `npm audit`.

15. Decisiones de diseño

Las respuestas no se purgan al cerrarse una rama. Cambiar de opinión no debe destruir trabajo. El costo es que hace falta evaluación transitiva; el beneficio, que explorar el cuestionario es seguro.

El contenido se genera, no se escribe. 112 preguntas mantenidas a mano se desincronizan del documento fuente. Con el generador, el criterio vive en un solo archivo y el contenido se puede reaplicar cuando el Excel cambie.

Ids desacoplados del número visible. Renumerar una dimensión no debe invalidar las respuestas guardadas ni el feedback ya recogido.

El PDF se recorre desde los datos. Ver [sección 8](#).

Las reglas de visibilidad son compartidas. Una sola definición de “esta pregunta aplica”, usada por el formulario y por el documento.

El feedback no viaja con las respuestas. `QuestionFeedback` manda el identificador de la pregunta y el comentario, nunca lo que la persona escribió.

Nada del contenido de la ficha sale del navegador. Es un compromiso con quien la usa, y está declarado en la portada.

16. Cómo extender la herramienta

Cambiar el texto de una pregunta

En el Excel del Drive → reextraer → regenerar:

```
python3 docs/fta-gen/extraer-excel.py      # requiere openpyxl
node docs/fta-gen/generar-sections.mjs
```

Revisar los avisos y el diff de `sections.ts`.

Agregar o quitar una pregunta

Igual que arriba, pero **la numeración por posición se corre**, y con ella los `id`.

Consecuencias a revisar:

- Las reglas de `CONDICIONES` que apunten a números posteriores.
- `PREGUNTAS_CLAVE` en `question-types.ts` — el generador avisa si quedan colgando.
- Las respuestas guardadas de esa dimensión dejan de corresponder.

Agregar contenido internacional

En el Excel, una fila con la misma ID y `Versión = Internacional`. El generador la empareja y emite `overrides.internacional`. Para una pregunta **exclusiva** de un contexto, marcar `soloContexto` a mano tras generar (o extender el generador).

Agregar un tipo de campo

1. Añadir el literal a `QuestionType` en `question-types.ts`.
2. Añadir el caso en `renderQuestionInput` de `transparency-tool.tsx`.
3. Añadir el caso en el componente `Dato` de `preview-ficha.tsx` — **si no, la respuesta no aparece en la ficha.**
4. Enseñarle el tipo al generador (TIP0 o la inferencia).

Agregar una pregunta a la encuesta

`src/data/survey.ts`. Los `id` **no son libres**: `/api/survey` los mapea a columnas fijas de `tool_survey`. Agregar una pregunta implica migración + actualizar el mapeo de la ruta.

Activar el subconjunto de IA generativa

`IA_GEN_QUESTION_ID` en `question-types.ts` está en `null`, así que las preguntas con `iaGen` se muestran siempre. Al definir el `id` de la pregunta filtro, se activa la bifurcación. Falta que el equipo de contenido defina esa pregunta (consulta 4 en [../fta-gen/consultas-equipo.md](https://github.com/AdolfoIbanez/fta-gen/consultas-equipo.md)).

Herramienta desarrollada por **GobLab UAI**, Escuela de Gobierno de la Universidad Adolfo Ibáñez, con el apoyo de la Agencia Nacional de Investigación y Desarrollo (ANID) — Subdirección de Investigación Aplicada / Concurso IDeA I+D 2023, proyecto ID23I10357.